ALGORITHMS ANALYSIS AND DESIGN

Sorting Algorithms

Agenda

- Insertion Sort
- Selection Sort
- Bubble Sort
- Assignment

Insertion Sort

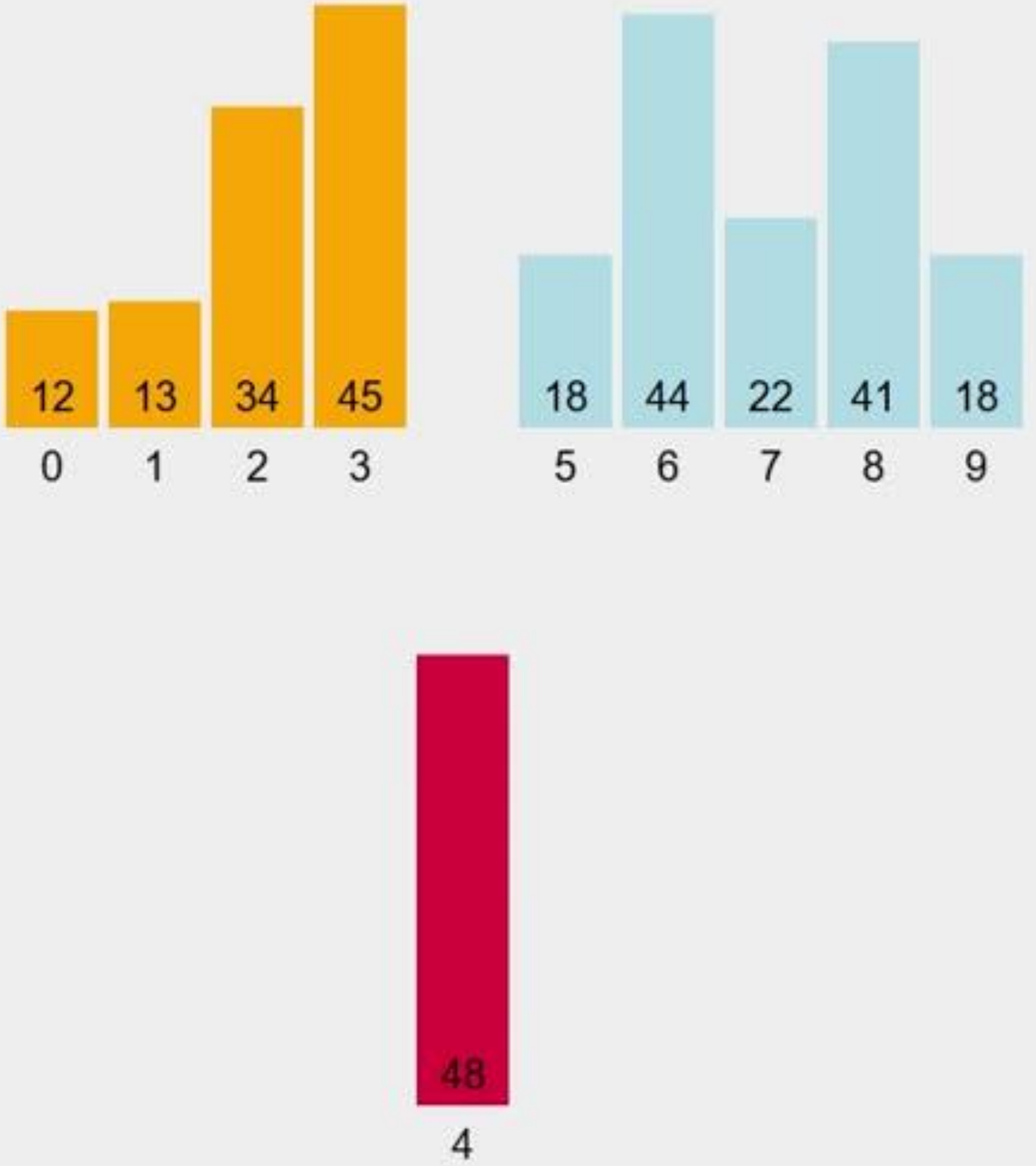
- ***Insertion sort*** is a simple sorting algorithm that works by iteratively ***inserting each element of an unsorted list into its correct position*** in a sorted portion of the list

- It is like sorting playing cards in your hands.
  - You split the cards into two groups: the sorted cards and the unsorted cards.
  - Then, you pick a card from the unsorted group and place it in the correct position in the sorted group.

- **Think:**
  - When analyzing a sorting algorithm, what exactly do we mean by best case, average case, and worst case scenarios for sorting a set of elements?
  - In other words, what determines whether a particular input is considered a best, average, or worst case for that sorting algorithm?

Insertion Sort: How It Works

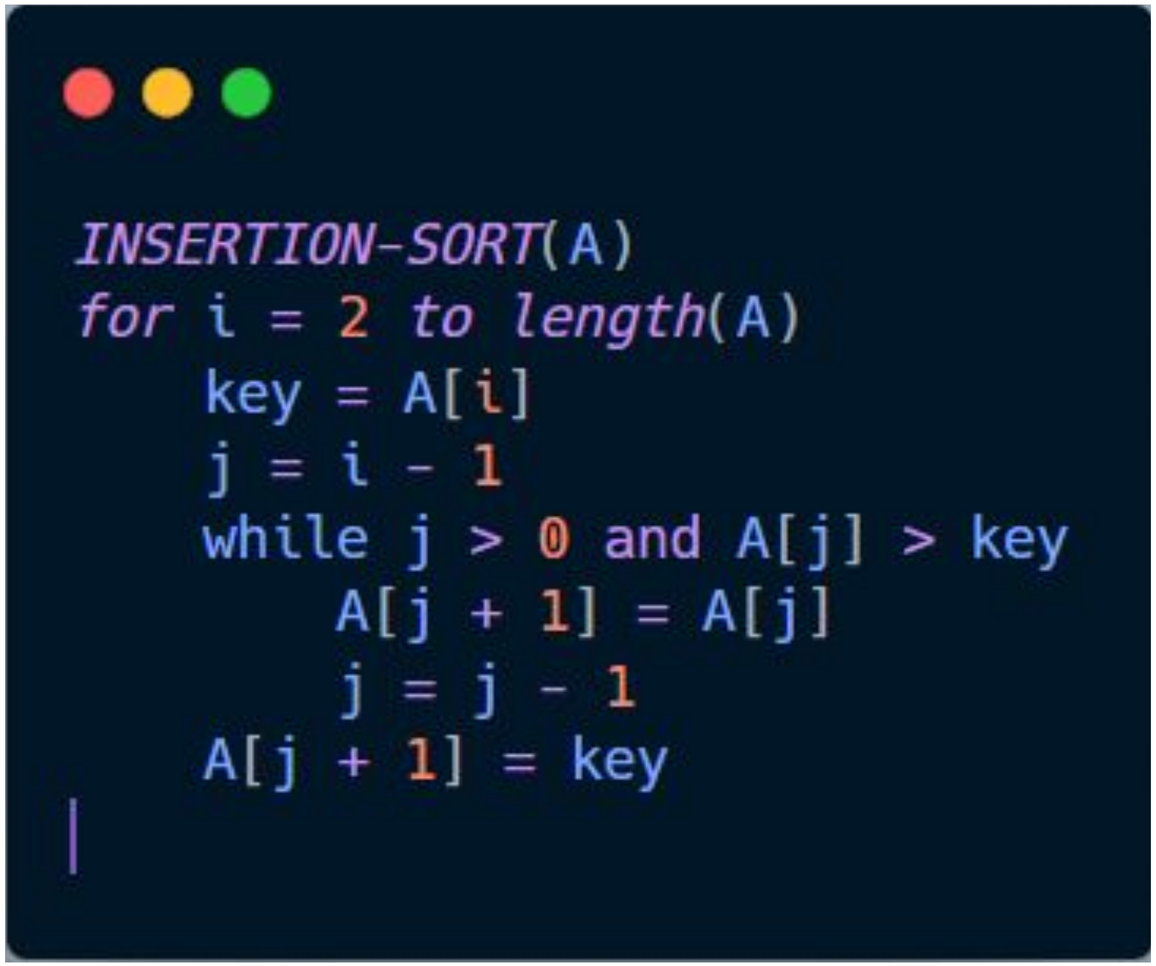
1. Start from the second element.
2. Compare it with the elements before it.
3. Move larger elements one position to the right.
4. Insert the element into its correct position.
5. Repeat for all elements.



Complexity

- Best case: ***O(n)***, If the list is already sorted, where n is the number of elements in the list.
- Average case: ***O(n²)***, if the list is randomly ordered
- Worst case: ***O(n²)***, if the list is in reverse order

Insertion Sort - Psudo code



```
INSERTION-SORT(A)
for i = 2 to length(A)
    key = A[i]
    j = i - 1
    while j > 0 and A[j] > key
        A[j + 1] = A[j]
        j = j - 1
    A[j + 1] = key
```

Insertion Sort – Sheet 2 questions

- Q1: Rewrite the INSERTION-SORT procedure to sort into monotonically decreasing instead of monotonically increasing order.

Insertion Sort – Sheet 2 questions

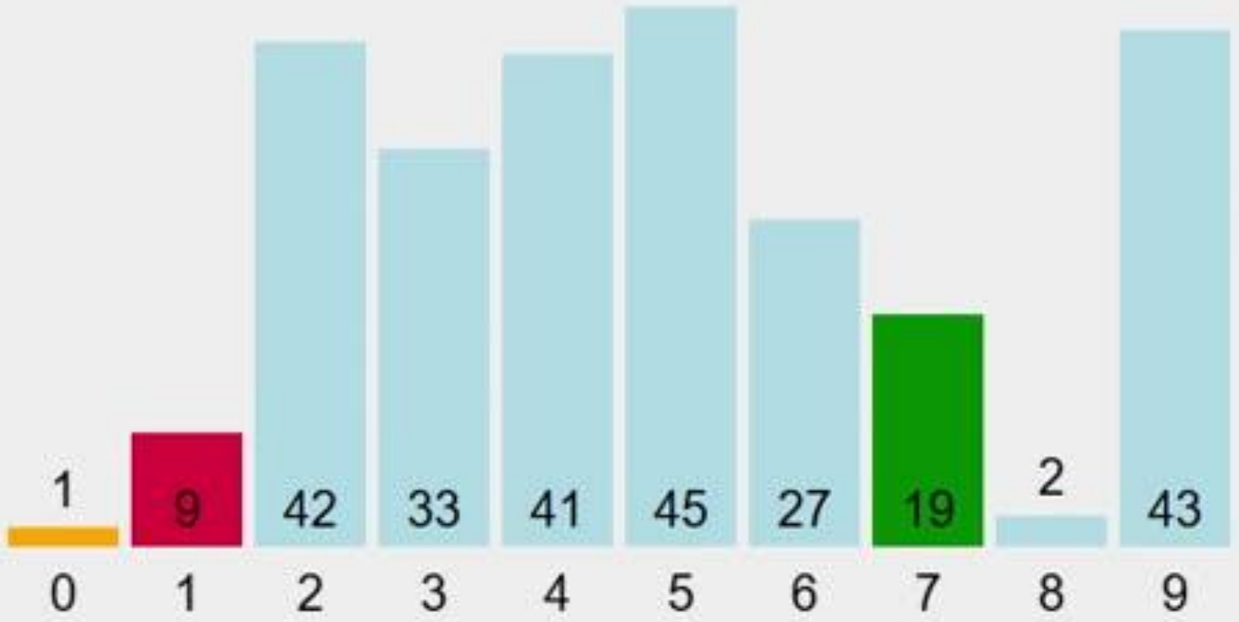
- Q4: Illustrate the operation of insertion sort on an array initially containing the sequence
  - {3; 41; 52; 26; 38; 57; 9; 49}

Selection Sort

- **Selection Sort** is a **comparison-based** sorting algorithm.
- It sorts an array by repeatedly selecting the smallest (or largest) element from the unsorted portion and swapping it with the first unsorted element.
- This process continues until the entire array is sorted.

Selection Sort: How It Works

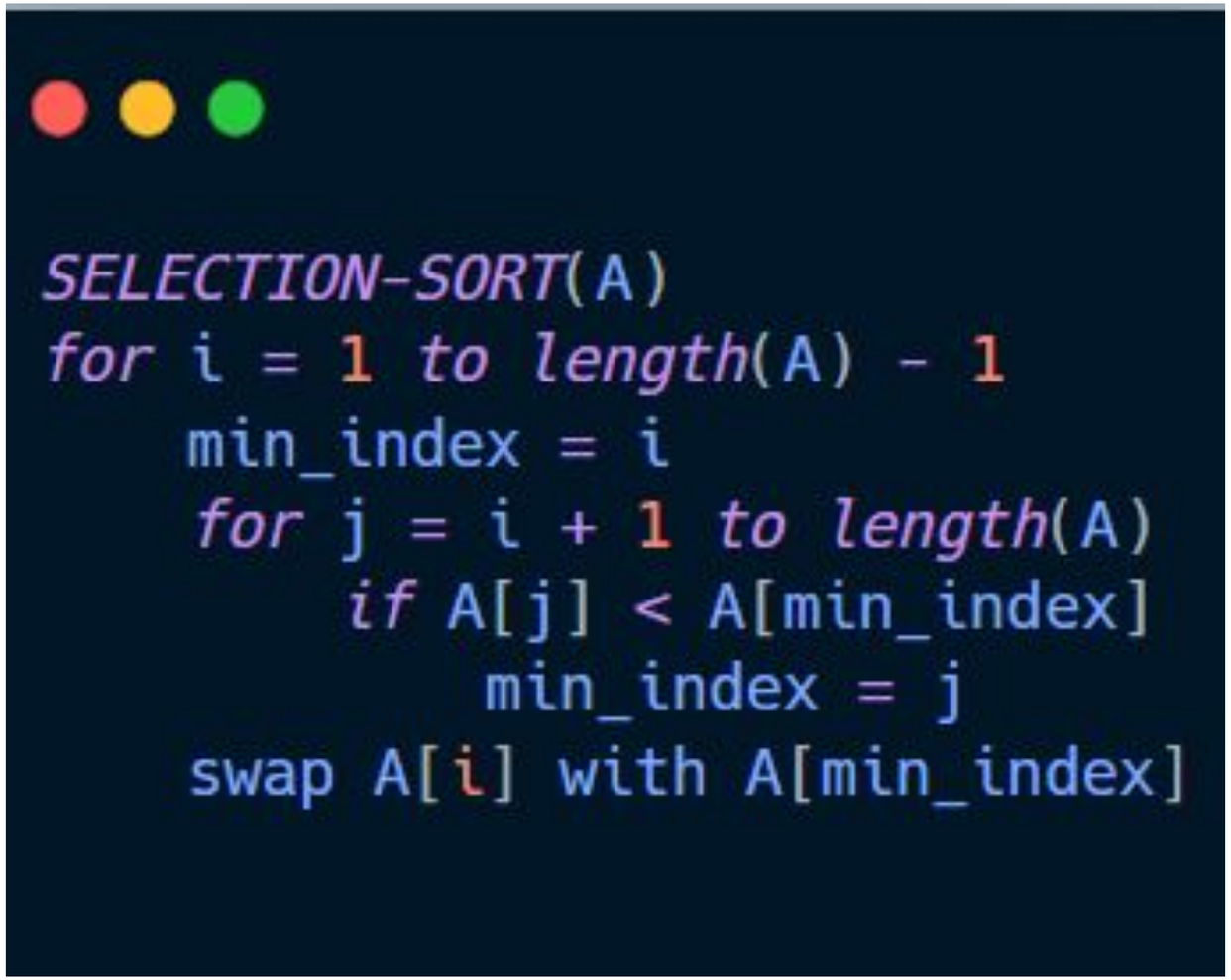
1. Start from the first element.
2. Find the smallest element in the unsorted part of the array.
3. Swap it with the first element of the unsorted part.
4. Move the boundary of the sorted part one step to the right.
5. Repeat until the entire array is sorted.



Complexity

- Best case: ***O(n²)***, even if the list is already sorted — selection sort always scans the entire unsorted part to find the minimum.
- Average case: ***O(n²)***, when the list is randomly ordered.
- Worst case: ***O(n²)***, when the list is in reverse order.

Selection Sort - Psudo code



```
SELECTION-SORT(A)
for i = 1 to length(A) - 1
    min_index = i
    for j = i + 1 to length(A)
        if A[j] < A[min_index]
            min_index = j
    swap A[i] with A[min_index]
```

Selection Sort – Sheet 2 questions

- Q3: Consider sorting n numbers stored in array A[1 : n] by first finding the smallest element of A[1 : n] and exchanging it with the element in A[1]. Then find the smallest element of A[2 : n], and exchange it with A[2]. Then find the smallest element of A[3 : n], and exchange it with A[3]. Continue in this manner for the first n - 1 elements of A.
  - Why does it need to run for only the first n - 1 elements, rather than for all n elements?
  - Give the worst-case running time of selection sort in θ notation.
  - Is the best-case running time any better?

Selection Sort – Sheet 2 questions

- Important Definition
  - A ***loop invariant*** is a condition (or statement) that is true before and after every iteration of a loop.
  - It's a logical way to prove the correctness of an algorithm — meaning, it helps us show that when the loop finishes, the algorithm has done what we expect.

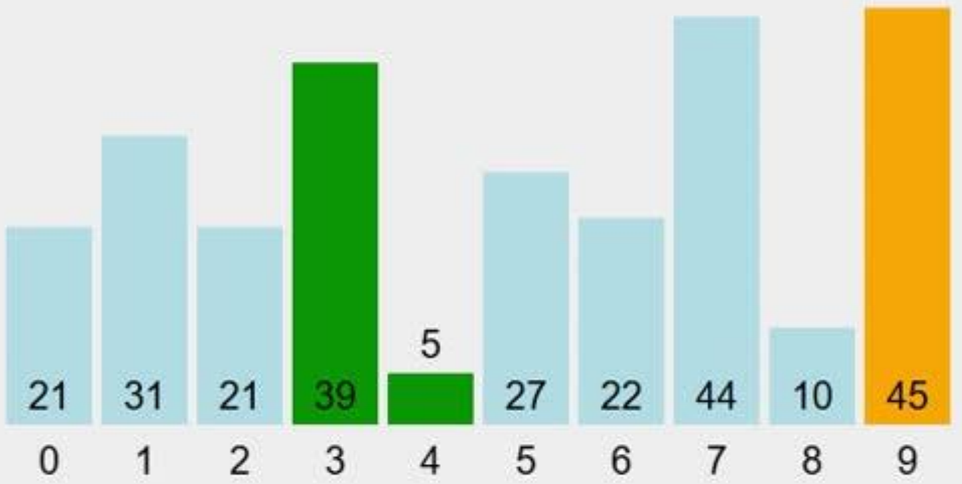
- Continue Q3:
- **What loop invariant does this algorithm maintain?**
  - Initialization (before first iteration):
    - i = 1, so A[1 .. 0] is empty — that's trivially sorted
  - Maintenance (during each iteration):
    - The algorithm finds the smallest element from A[i .. n] and swaps it with A[i],
    - so now A[1 .. i] is sorted and contains the i smallest elements
  - Termination (after loop ends):
    - When i = n, the invariant tells us A[1 .. n - 1] is sorted,
    - and since only one element (A[n]) remains, the entire array is sorted

Bubble Sort

- Bubble Sort is the simplest sorting algorithm that works by repeatedly swapping the adjacent elements if they are in the wrong order.
- This algorithm is not suitable for large data sets as its average and worst-case time complexity are quite high.

Bubble Sort: How It Works

1. Start from the first element.
2. Compare each pair of adjacent elements.
3. If the current element is greater than the next one, swap them.
4. Continue this process until the end of the array — the largest element will “bubble up” to the end.
5. Move the boundary of the unsorted part one step to the left (ignore the last sorted elements).
6. Repeat the process for the remaining unsorted part until the entire array is sorted.



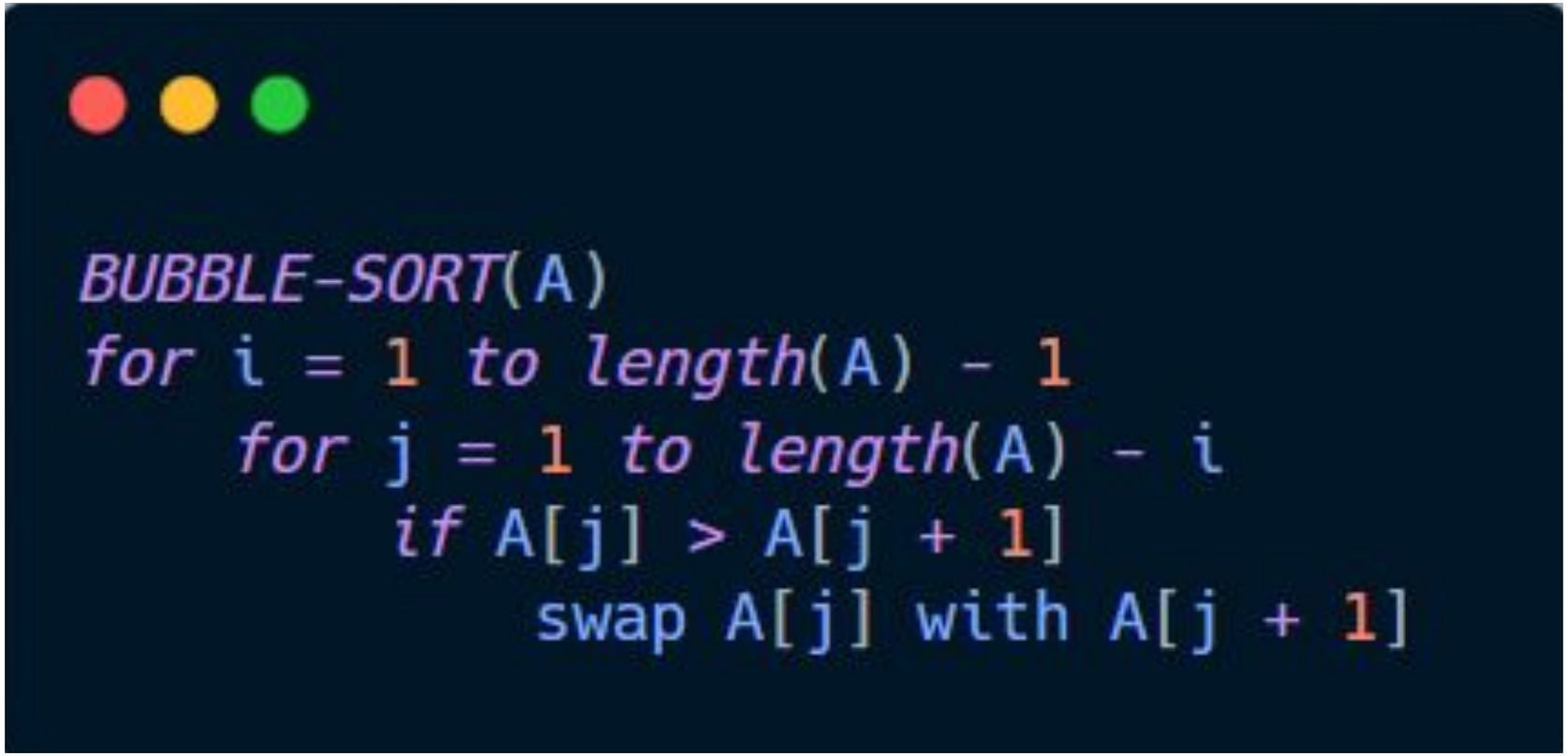
Bubble Sort: Cont, How It Works

Complexity

- Best case: ***O(n²)*** for the basic version , because it still performs all comparisons even if the list is sorted.
- Average case: ***O(n²)***, when the list is randomly ordered.
- Worst case: ***O(n²)***, when the list is in reverse order and every pair must be swapped.

- **Think**:
  - Can we optimize bubble sort to get a better time complexity? How?
  - Why is bubble sort considered worse than selection sort, although they have the same exact complexity? (hint: **Compare Constant Factor**)

Bubble Sort - Psudo code



```
BUBBLE-SORT(A)
for i = 1 to length(A) - 1
    for j = 1 to length(A) - i
        if A[j] > A[j + 1]
            swap A[j] with A[j + 1]
```

Bubble Sort – Sheet 2 questions

- Q6: Bubblesort is a popular, but inefficient, sorting algorithm. It works by repeatedly swapping adjacent elements that are out of order. The procedure BUBBLESORT sorts array A [1: n].
  - What is the worst-case running time of BUBBLESORT?
  - How does it compare with the running time of INSERTION-SORT?

General – Sheet 2 questions

- Q2: Consider the problem of adding two n-bit binary integers a and b, stored in two n-element arrays A[0:n-1] and B[0:n-1], where each element is either 0 or 1, a = sum from i=0 to n-1 of A[i]·2^i, and b = sum from i=0 to n-1 of B[i]·2^i. The sum c = a + b of the two integers should be stored in binary form in an (n+1)-element array C[0:n], where c = sum from i=0 to n of C[i]·2^i.
  - Write a procedure ADD-BINARY-INTEGERS that takes as input arrays A and B, along with the length n, and returns array C holding the sum.

🎯 LeetCode Assignment

🧩 Tasks

- Create account on LeetCode
- Solve **10 problems** → 🟢 9 Easy + 🟡 1 Medium
- 👤 Individual work (no collaboration)

📹 Submission (Screen Recording)

- Show **profile + solved problems**
- Explain **1 problem** → 💡 logic + 💻 code + 🤔 approach

📤 Upload

- Upload to Google Drive
- Submit link via the task form (To be provided later)

⏰ Deadline: April 15, 2026 | 11:59 PM

THANK YOU